Programowanie współbieżne i rozproszone

mgr inż. Marcin Mrukowicz

Instrukcja laboratoryjna III

Komunikaty międzyprocesowe w systemie UNIX i jego sukcesorach (Linux i OS X)

Przekazywanie komunikatów (ang. message passing) to podstawowa obok pamięci współdzielonej (ang. shared memory) technika komunikacji międzyprocesowej (Inter process communication, IPC). W obrębie przekazywania komunikatów możemy rozpatrywać komunikaty międzyprocesowe. Najogólniej rzecz ujmując, komunikaty takie pozwalają przekazywać informację pomiędzy procesami w celu ich synchronizacji. Przykładowo proces może poinformować inne zainteresowane procesy, że wykonał już jakieś zadanie albo że zwalnia dany zasób współdzielony. Z kolei wspomniane zainteresowane procesy mogą wstrzymać swoje wykonanie do czasu uzyskania komunikatu od innego procesu.

Bardzo starym sposobem na uzyskanie komunikacji międzyprocesowej jest wykorzystanie grupy funkcji: *kill* i *signal*. Funkcja *kill* posiada lekko myląca nazwę: ogólnie służy ona do wysłania komunikatu do procesu o podanym pid. Domyślnie był to rzeczywiście komunikat nakazujący procesowi zakończenie wykonania, jednakże funkcja ta nie musi zakończyć wykonania procesu. Funkcja *signal* umożliwia z kolei napisanie bloku programu, który zostanie wykonany po dotarciu do procesu danego sygnału. Funkcję taką możemy nazwać po angielsku signal handler. Jest to rozwiązanie przypominające znany z programowania obiektowego wzorzec projektowy obserwator.

W terminologii systemu UNIX komunikaty nazywamy sygnałami. W systemie tym istnieje kilkanaście podstawowych sygnałów, m.in.:

- SIGINT (signal interrupt) - jest to sygnał, który nakazuje procesowi zakończenie wykonania. Zwykle pozwala mu się najpierw zwolnić swoje zasoby (w tym otwarte pliki) i dopiero zakończyć wykonanie. Jest to sygnał, który wysyłamy procesowi, naciskając w terminalu ctrl + c (w niektórych niszowych już dziś systemach delete albo break).
- SIGKILL (signal kill) - jest to sygnał, który nakazuje procesowi natychmiast zakończyć działanie. Sygnał ten nie może być zignorowany i proces nie może również zwolnić swoich zasobów. Sygnał ten raczej jest wykorzystywany przez system operacyjny. Przykładem jest zakończenie wykonania zapętlonych procesów.
- SIGTSTP (signal terminal stop) - jest to sygnał, który nakazuje procesowi zatrzymanie wykonywania się (proces jednak nie zostanie usunięty z liczby wykonywanych procesów, a jedynie wstrzymany). Sygnał ten wysyłamy do procesu, naciskając w terminalu ctrl + z.
- SIGCONT (signal continue) - jest to sygnał, który nakazuje procesowi zacząć ponownie się wykonywać, po wstrzymaniu np. za pomocą funkcji SIGTSTP.
- SIGSEGV (signal segmentation violation) - jest to sygnał, który zostanie wysłany do procesu, gdy zażąda on dostępu do niedostępnej dla niego segmentu pamięci operacyjnej.
- SIGFPE (signal floating point exception) - jest to sygnał wysyłany do procesu, który chciał wykonać niedozwoloną operację arytmetyczną (np. dzielenie przez 0).
- SIGUSR1, SIGUSR2 (signal user 1, signal user 2) - są to sygnały użytkownika (zdefiniowane przez użytkownika); **domyślnie powodują one zakończenie działania procesu.** Jednakże ich przeznaczeniem jest **zdefiniowanie własnych komunikatów i nadpisanie domyślnego handleru.**

Co ważne każdy proces ma domyślnie zdefiniowane signal handlers, obsługujące powyższe sygnały.

Pomocna w synchronizacji procesów jest funkcja *pause*, która wstrzymuje wykonanie programu aż do czasu otrzymania dowolnego komunikatu (sygnału). Funkcja ta różni się od *wait*, tym, że oczekujemy na dowolny sygnał systemowy, wysłany przez dowolny proces, podczas gdy *wait* wstrzymuje wykonanie procesu wyłącznie do czasu zakończenia wykonywania przez jego własny proces potomny.

Kolejka komunikatów w systemie UNIX i jego sukcesorach (Linux i OS X)

Kolejka komunikatów (ang. message queve) w przeciwieństwie do sygnałów może być utworzona przed utworzeniem komunikujących się za jej pomocą procesów. Ponadto możliwa jest komunikacja asynchroniczna za jej pomocą (tzn. jeden proces może wysłać komunikat, a jego adresat może odczytać go później). Procesy nie muszą także znać swoich pid, wystarczy że potrafią „podłączyć się" do wspólnej kolejki.

W systemie UNIX istnieje grupa funkcji: *msgget, msgsnd, msgrcv* i *msgctl*. Ogólnie wiadomość musi mieć zdefiniowany typ, po czym następuje opcjonalnie treść wiadomości. Najprostsza struktura wiadomości to przykładowo:

```
struct msgbuf {
long mtype; /* typ wiadomości */
char *mtext; /* wiadomość */
}
```

Możliwe jest zbudowanie bardziej złożonej struktury:

```
struct my_msgbuf {
long mtype;   /* typ wiadomości */
short mshort; /* dowolne pole */
char mchar[10]; /* dowolne pole */
}
```

Wzajemne wykluczanie (ang. mutual exclusion)

Niejednokrotnie w celu zachowania spójności danych, współdzielonych przez procesy, bądź wymuszenia wykonania przez procesy zadań w pewnej z góry określonej kolejności, istnieje potrzeba zastosowania wzajemnego wykluczania. Blok kodu, w którym proces uzyskuje wyłączny dostęp do zasobu nazywamy **sekcją krytyczną** (ang. **critical section**).

Najprostszym sposobem na zapewnienie wzajemnego wykluczania jest zastosowanie tzw. **mutexu (od ang. mutual exclusion)**. Jest to w najprostszym przypadku **zmienna binarna**, współdzielona przez procesy, która blokuje dostęp do jakiegoś zasobu (np. jeżeli jest ustawiona na wartość false, w języku C: 0) lub pozwala na dostęp do niego (np. jeżeli jest ustawiona na wartość true, w języku C: 1).

Uogólnieniem mutexu jest **semafor (ang. semaphore)**. Jest to w najprostszej implementacji zmienna całkowita, która określa maksymalną liczbę procesów, które mogą korzystać z danego zasobu. Przykładowo ustawiając semafor początkowo na wartość 5, ustalamy, że maksymalnie 5 procesów może mieć dostęp do danego zasobu. Gdy wątek dostaje dostęp do zasobu, to dekrementuje semafor, a podczas gdy zwalnia ten zasób, to z kolei inkrementuje semafor. Gdy wartość semaforu osiągnie 0, to nie pozwalamy już kolejnym procesom na dostęp do zasobu. Dopiero gdy jakiś proces zwolni zasób, to zwiększy wartość semafora (np. na 1) i znowu jakiś proces, oczekujący na zasób, może mieć do niego dostęp. W teorii, gdy wszystkie procesy zwolnią

zasób, to semafor ponownie uzyska wartość 5. **Semafor, któremu ustawimy maksymalną liczbę procesów na 1, redukuje się do mutexu.**

Kontrola zasobów współdzielonych z poziomu konsoli

Jeżeli chcielibyśmy zaalokować pamięć współdzieloną, która już istnieje, otrzymamy błąd: Segmentation fault. Pamięć współdzielona domyślnie nie zostanie zwolniona, w tym po ponownym uruchomieniu komputera, jeżeli procesy, które ją utworzyły tego nie zrobią.

W celu podglądnięcia aktualnie wykorzystywanych zasobów współdzielonych, można użyć polecenia:

```
ipcs
```

Jeżeli chcemy usunąć jakiś zasób, który nie został zwolniony (na skutek błędów implementacyjnych), możemy wykorzystać polecenie o ogólnej sygnaturze, przy czym zadziała ono, dopiero jeżeli nie ma procesu, który używa dany zasób (nattch – number of attachement jest równe 0):

```
ipcrm <msg | sem | shm>  <IPC ID>
```

Żeby ustalić pid odpowiednich procesów można użyć polecenia:

```
 ipcs -mp
```

Następnie procesy, podłączone do segmentu pamięci współdzielonej należy zatrzymać, choćby za pomocą polecenia *kill*.
UWAGA! Procesy systemowe, bądź inne programy również korzystają z pamięci współdzielonej i warto sprawdzić jaki proces ją alokuje.
Przykłady:

```
marcin@marcin-virtual-machine:~$ ipcs

------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x00000000 7503911    marcin     600        16384      1          dest
0x00000000 7503914    marcin     600        524288     2          dest
0x00000000 7503915    marcin     600        8355840    2          dest
0x00000000 7503925    marcin     600        67108864   2          dest
0x00000000 7471158    marcin     777        8294400    2          dest
0x00000000 7503930    marcin     600        524288     2          dest
0x00000000 7503931    marcin     600        1916928    2          dest
0x00000000 7471167    marcin     600        524288     2          dest
0x0001e240 7536674    marcin     666        220        2

------ Semaphore Arrays --------
key        semid      owner      perms      nsems
```

```
marcin@marcin-virtual-machine:~$ ipcs -mp

------ Shared Memory Creator/Last-op PIDs --------
shmid      owner      cpid       lpid
7503911    marcin     133675     133716
7503914    marcin     133675     130921
7503915    marcin     133675     130921
7503925    marcin     131450     130921
7471158    marcin     130921     133740
7503930    marcin     133916     133924
7503931    marcin     133675     130921
7471167    marcin     131450     133740
7536674    marcin     135525     135551
```

```
marcin@marcin-virtual-machine:~$ ps -p 133675
    PID TTY          TIME CMD
 133675 ?        00:00:19 gnome-shell
```

```
marcin@marcin-virtual-machine:~$ ps -p 133716
    PID TTY          TIME CMD
 133716 ?        00:00:00 ibus-daemon
marcin@marcin-virtual-machine:~$ ps -p 131450
    PID TTY          TIME CMD
 131450 ?        00:00:13 snap-store
marcin@marcin-virtual-machine:~$ ps -p 135525
    PID TTY          TIME CMD
 135525 pts/1    00:00:00 chat1
marcin@marcin-virtual-machine:~$ ps -p 135551
    PID TTY          TIME CMD
 135551 pts/2    00:00:00 chat2
```

```
ubuntu@Latitude-5540:~$ ipcs

------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x000004d2 5          ubuntu     666        0            0

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status

------ Semaphore Arrays --------
key        semid      owner      perms      nsems
```

```
ubuntu@Latitude-5540:~$ ipcrm -Q 0x000004d2
```

1. Przepisz poniższy program i nazwij go: *signals_hello_world.c*:

```c
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

// jest to funkcja (signal handler)
// która wykona się po otrzymaniu
// danego sygnału
// w przypadku programu hello world
// nie wykonujemy żadnej akcji
// zdefiniowanie tej funkcji
// zapobiega jednak zakończeniu
// działania procesu (domyślne zachowanie po otrzymaniu sygnału SIGUSR1)
void parent_have_executed(int signum) {}

int main() {
    int pid;

    if ((pid = fork())== -1) {
        perror("fork() failed");
        exit(1);
    }

    if (pid == 0) {
        // proces dziecka
        // definiujemy signal handler dla sygnału
        // SIGUSR1, jest to funkcja parent_have_executed
        signal(SIGUSR1, parent_have_executed);
        // wstrzymujemy działanie programu do otrzymania
        // dowolnego sygnału
        pause();
        // kontynnujemy wykonanie programu
        printf("World!\n");
        fflush(stdout);
    }
    else {
        // proces rodzica
        // wykonujemy program rodzica
        printf("Hello, ");
        fflush(stdout);
        // wysyłamy sygnał SIGUSR1 do procesu
        // potomnego, zmienna pid przechowuje
        // identyfikator procesu potomnego.
```

```
        kill(pid, SIGUSR1);
        // proces rodzicielski czeka na zakończenie
        // działania przez proces potomny
        wait(NULL);
    }

    exit(0);
}
```

Uruchom powyższy program kilka razy.

- **Czy za każdym razem uzyskiwane jest oczekiwane wyjście? Czy program za każdym razem zachowuje się tak samo?**

W przypadku procesów, które wykonują niewiele instrukcji istnieje możliwość, że jeden z procesów nie zdąży w pełni się zainicjować, podczas gdy drugi proces wykona już większość swojej logiki, co może powodować nieoczywiste błędy. W przykładzie powyżej, ponieważ funkcja signal dowiązuje handler dopiero przy uruchomieniu programu istnieje możliwość, że zanim proces potomny **zdąży nadpisać domyślny handler sygnału** (**który domyślnie powoduje zakończenie działania programu**), **proces rodzicielski zdąży już wysłać ten sygnał i co za tym idzie zakończyć działanie procesu potomnego**. Powoduje, to że z jednej strony program nie zawiesza się w sposób widoczny w terminalu i prawidłowo kończy działanie (ponieważ rodzic sam wymusił na procesie potomnym zakończenie działania, wait nie spowoduje zawieszenia programu), jednak oczywiście nie realizuje w pełni swojej logiki, bo proces potomny nie wykonuje właściwie żadnej instrukcji.

Teoretycznie pewnym rozwiązaniem tego problemu jest uśpienie na krótko procesu rodzicielskiego, aby proces potomny na pewno zdążył się zainicjować. Logikę tę realizuje program *signals_hello_world_sleep.c*

```c
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void parent_have_executed(int signum) {}

int main()
{
    int pid;
    if ((pid = fork()) == -1)
    {
        perror("fork() failed");
        exit(1);
    }
    if (pid == 0)
    {
        signal(SIGUSR1, parent_have_executed);

        pause();

        printf("World!\n");
        fflush(stdout);
    }
    else
    {
        printf("Hello, ");
        fflush(stdout);

        // istnieje możliwość, że proces dziecka nie zdąży
        // zainicjować nowego handleru sygnału SIGUSR1 a
        // domyślnie sygnał SIGUSR1 zabija proces
        // musimy zatem upewnić się, że dziecko zdąży się uruchomić
        // i nadpisać handler sygnału SIGUSR1
        // najprostsze rozwiązanie, to uśpić na chwilę proces rodzica
        usleep(10);

        kill(pid, SIGUSR1);
```

```
        wait(NULL);
    }
    exit(0);
}
```

- **Czy Twoim zdaniem to rozwiązanie gwarantuje poprawność działania programu?**

Poniżej zaprezentowano rozwiązanie wykorzystujące semafor.

```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <semaphore.h>
#include <errno.h>
#include <sys/types.h>
#include <fcntl.h>

void parent_have_executed(int signum) {}

int main()
{
    // wskaźnik na semafor
    sem_t *mutex_sem;

    // tworzymy semafor, obydwa procesy będą mogły go zarówno blokować, jak i zwalniać
    // inicjujemy semafor wartością zero, czyli jest on zablokowany
    if ((mutex_sem = sem_open("/sem-mutex", O_CREAT, 0660, 0)) == SEM_FAILED) {
        perror ("sem_open"); exit (1);
    }

    int pid;
    if ((pid = fork()) == -1)
    {
        perror("fork() failed");
        exit(1);
    }
    if (pid == 0) {
        signal(SIGUSR1, parent_have_executed);

        // dziecko inkrementuje wartość semafora, na co czeka obecnie rodzic
        // innymi słowy dziecko zwalnia semafor
        sem_post(mutex_sem);

        pause();

        printf("World!\n");
        fflush(stdout);
    }
    else {
        printf("Hello, ");
        fflush(stdout);

        // istnieje możliwość, że proces dziecka nie zdąży
        // zainicjować nowego handleru sygnału SIGUSR1 a
        // domyślnie sygnał SIGUSR1 zabija proces
        // musimy zatem upewnić się, że dziecko zdąży się uruchomić
        // i nadpisać handler sygnału SIGUSR1
        // bardziej złożone rozwiązanie to zrobić to za pomocą semafora
        // tutaj rodzic czeka, aż dziecko zinkrementuje semafor
        // czyli czekamy na odblokowanie semafora
        sem_wait(mutex_sem);

        kill(pid, SIGUSR1);

        wait(NULL);

        // usuwamy utworzony semafor
        sem_unlink("/sem-mutex");
    }
    exit(0);
}
```

Program wykorzystał sygnały do synchronizacji standardowego wyjścia procesu potomnego i rodzicielskiego. Jednocześnie w celu zagwarantowania, że proces potomny zdąży się prawidłowo zainicjować powstała potrzeba zastosowania semafora.

- **Czy potrafisz wytłumaczyć, dlaczego nie można było użyć funkcji *wait* w procesie potomnym?**
- **Spróbuj zainicjować semafor wartością 1 (prawda). Jaki jest efekt działania programu?**
- **Dodaj instrukcję sleep(10) po wait. Przejdź do katalogu /dev/shm i wpisz komendę ls. Ponownie wpisz komendę ls po zakończeniu działania programu. Co oberwujesz?**
- **Zmodyfikuj program w ten sposób, żeby jedynie za pomocą semafora synchronizować wypisanie komunikatu „Hello world!" przez procesy.**

2. Przepisz poniższy program:

```c
#include <sys/msg.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

// struktura danych do przechowywania wiadomości
struct mesg_buffer {
    long mesg_type; // typ wiadomości
    char mesg_text[100]; // treść wiadomości
};

int main() {
    // tworzymy unikalny identyfikator dla kolejki wiadomości
     key_t key = 1234;

    // pobieramy id kolejki do odczytu i zapisu, jeżeli
    // nie istnieje tworzymy ją
    int msgid = msgget(key, 0666 | IPC_CREAT);
    int status;

    if ((status = fork()) == -1) {
        printf("%s", "fork failed");
        exit(1);
    }

    if (status == 0) {
        struct mesg_buffer msg;

       // dziecko odbiera pierwszą z brzegu wiadomość z kolejki
       int status = msgrcv(msgid, &msg, sizeof(msg.mesg_text), 0, 0);
        if (status == -1) {
            perror("msgrcv failed");
            exit(EXIT_FAILURE);
        }
```

```
        // formalnie sprawdzamy, czy do dziecka dotarła
        // oczekiwana wiadomość od rodzica
        if (strcmp(msg.mesg_text, "wrote") == 0) {
            printf("%s", "world!\n");
        }

        // usuwamy kolejkę wiadomości
        status = msgctl(msgid, IPC_RMID, NULL);

        if (status == -1) {
            perror("msgctl failed");
            exit(EXIT_FAILURE);
        }
    } else {
        printf("%s", "Hello ");
        struct mesg_buffer msg;

        // ustawiamy typ wiadomości
        msg.mesg_type = 1;
        // ustawiamy wartość tekstu wiadomości na wrote
        memcpy(msg.mesg_text, "wrote", 5);

        // wysyłamy wiadomość, jeżeli kolejka nie jest przepełniona (tzn. może mieć limit
wiadomości)
        // będziemy domyślnie czekać, aż pojawi się wolne miejsce w kolejce
        int status = msgsnd(msgid, &msg, sizeof(msg.mesg_text), 0);
        if (status == -1) {
            perror("msgsnd failed");
            exit(EXIT_FAILURE);
        }
        // rodzic, w tym miejscu kończy działanie, jeżeli udało się wysłać wiadomość
    }
}
```

- **Uruchom program kilkukrotnie, czy zapewnia on synchroniczne wypisanie Hello world! na ekran?**
- **Skomentuj przesłanie komunikatu przez rodzica. Jakie jest działanie programu?**
- **W konsoli wykonaj następujące polecenie: ipcrm -Q 0x000004d2 Co się stało? Czy potrafisz wytłumaczyć zachowanie programu?**
- **W wywołaniu funkcji *msgrcv* ustaw ostatni argument na IPC_NOWAIT. Ponownie uruchom program przy założeniu, że rodzic nie przesyła komunikatu. Co się stało?**
- **Zmodyfikuj program tak, aby rodzic wysłał do dziecka Hello, a dziecko do rodzica World!. Zaplanuj prawidłową kolejność przesłania komunikatów. Wypisz na ekran tylko wartości tekstów komunikatów, tak, aby wypisano Hello World!**

3. Przepisz poniższe programy, nazwij je odpowiednio *chat1.c* i *chat2.c:*

```
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
```

```c
#include <sys/types.h>
#include <unistd.h>

#define FILLED 0
#define Ready 1
#define NotReady (-1)


// struktura w której
// będziemy przechowywać
// bufor, identyfikatory procesów i stan
// gotowości procesów
struct memory {
    char buff[100];
    int status, pid1, pid2;
};

// wskaźnik, w którym będziemy
// przechowywać strukturę
struct memory* shared_memory;

void handler(int signum)
{
    // nadpisujemy handler dla sygnału
    // SIGUSR1, będzie to wiadomość
    // od drugiego użytkownika czatu
    // wiadomość będzie wysyłał inny proces
    if (signum == SIGUSR1) {
        printf("Received User2: ");
        puts(shared_memory->buff);
    }
}

int main()
{
    // pobieramy identyfikator procesu
    // proszę zwrócić uwagę, że nie wywołano
    // fork; jednak ponieważ każdy proces ma pid
    // to możemy je odczytać
    int pid = getpid();

    // identyfikator bloku pamięci
    int shmid;

    // arbitralnie dobrany identyfikator bloku pamięci, wspólny
    // dla obydwu procesów
    int key = 12345;

    // tworzymy pamięć współdzieloną, do odczytu i zapisu
    shmid = shmget(key, sizeof(struct memory), IPC_CREAT | 0666);

    // dowiązujemy do wskaźnika shared_memory wcześniej zaalokowany blok pamięci
    shared_memory = (struct memory*)shmat(shmid, NULL, 0);

    // zapisujemy w strukturze identyfikator tego procesu
    // oraz status
    shared_memory->pid1 = pid;
    shared_memory->status = NotReady;

    // definiujemy handler dla sygnału SIGUSR1
    signal(SIGUSR1, handler);

    // pętla nieskończona
    while (1) {
        // czekamy aż drugi użytkownik podłączy się do czatu
        // i ustawi w pamięci współdzielonej własny pid
        while (shared_memory->status != Ready)
            continue;
        sleep(1);

        // pobieramy do bufora wiadomość od użytkownika 1 do użytkownika 2
        printf("User1: ");
        fgets(shared_memory->buff, 100, stdin);
        if (strcmp(shared_memory->buff, "q\n") == 0) break;
        // zmieniamy status,
        shared_memory->status = FILLED;

        // wysyłamy sygnał do drugiego procesu
```

```
        // ponieważ czekaliśmy na jego rozpoczęcie
        // mamy pewność, że sygnał na pewno dotrze do
        // adresata
        kill(shared_memory->pid2, SIGUSR2);
    }
    // zwalniamy dowiązanie wskaźnika shared_memory do pamięci współdzielonej
    shmdt((void*)shared_memory);
    // usuwamy blok pamięci współdzielonej
    // robi to tylko jeden proces
    shmctl(shmid, IPC_RMID, NULL);
    return 0;
}
```

Plik chat1.c

```
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/types.h>
#include <unistd.h>

#define FILLED 0
#define Ready 1
#define NotReady -1

// struktura w której
// będziemy przechowywać
// bufor, identyfikatory procesów i stan
// gotowości procesów
struct memory {
    char buff[100];
    int status, pid1, pid2;
};

// wskaźnik, w którym będziemy
// przechowywać strukturę
struct memory* shared_memory;



void handler(int signum)
{
    // nadpisujemy handler dla sygnału
    // SIGUSR2, będzie to wiadomość
    // od pierwszego użytkownika czatu
    // wiadomość będzie wysyłał inny proces
    if (signum == SIGUSR2) {
        printf("Received From User1: ");
        puts(shared_memory->buff);
    }
}

int main()
{
    // pobieramy identyfikator procesu
    // proszę zwrócić uwagę, że nie wywołano
    // fork; jednak ponieważ każdy proces ma pid
    // to możemy je odczytać
    int pid = getpid();

    // identyfikator bloku pamięci
    int shmid;

    // arbitralnie dobrany identyfikator bloku pamięci, wspólny
    // dla obydwu procesów
    int key = 12345;

    // tworzymy pamięć współdzieloną, do odczytu i zapisu
    shmid = shmget(key, sizeof(struct memory), IPC_CREAT | 0666);

    // dowiązujemy do wskaźnika shared_memory wcześniej zaalokowany blok pamięci
    shared_memory = (struct memory*)shmat(shmid, NULL, 0);

    // zapisujemy w strukturze identyfikator tego procesu
```

```
    // oraz status
    shared_memory->pid2 = pid;
    shared_memory->status = NotReady;

    // definiujemy handler dla sygnału SIGUSR2
    signal(SIGUSR2, handler);

    // pętla nieskończona
    while (1) {
        // ten proces rozpoczyna komunikację
        // więc nie czeka na proces 1
        sleep(1);

        // pobieramy do bufora wiadomość od użytkownika 2 do użytkownika 1
        printf("User2: ");
        fgets(shared_memory->buff, 100, stdin);
        if (strcmp(shared_memory->buff, "q\n") == 0) break;
        // ustawiamy status
        shared_memory->status = Ready;

        // wysyłamy sygnał do pierwszego procesu
        // który na niego oczekuje
        kill(shared_memory->pid1, SIGUSR1);

        // oczekujemy na odpowiedź od pierwszego procesu
        while (shared_memory->status == Ready)
            continue;
    }
    // zwalniamy dowiązanie wskaźnika shared_memory do pamięci współdzielonej
    shmdt((void*)shared_memory);
    return 0;
}
```

Plik chat2.c

- **Uruchom w jednym oknie terminalu proces *chat1* a w drugim oknie proces *chat2*. Co jest wynikiem działania programu?**
- **Zmodyfikuj program w ten sposób, aby usunąć pętle, które powodują, że procesy czekają nawzajem na odebranie wiadomości od siebie. Jak działa taki program?**
- **Wypowiedz się na temat wykorzystania jednego bufora. Jak oceniasz to rozwiązanie?**

4. Przepisz poniższe programy, nazwij je odpowiednio: chat_1_non_blocking.c i chat_2_non_blocking.c.

```
#include <ncurses.h> // biblioteka ncurses = new courses, da nam możliwość nieblokującego korzystania
// ze strumieni wejściowych
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>
#include <malloc.h>

#define BRIGHT_WHITE  15

char* local_buffer;

// struktura w której
// będziemy przechowywać
// 2 bufory, identyfikatory procesów i stan
// gotowości procesów
struct memory {
    char buff1[100];
    char buff2[100];
    int size, size2;
    int status, block, pid1, pid2;
};

// wskaźnik, w którym będziemy
// przechowywać strukturę
struct memory* shared_memory;
```

```
void print_and_clear_screen_if_needed(char* s) {
    int ok = printw(s);
    if (ok == ERR) {
        clear();
        printw(s);
    }
}


void handler(int signum)
{
    // nadpisujemy handler dla sygnału
    // SIGUSR2, będzie to wiadomość
    // od pierwszego użytkownika czatu
    // wiadomość będzie wysyłał inny proces
    if (signum == SIGUSR1) {
        // wiadomość od drugiego użytkownika
        // chcemy napisać w innym kolorze
        start_color();
        init_pair(1, COLOR_BLUE, COLORS > 15 ? 15 : COLOR_WHITE);
        init_pair(2, COLOR_BLACK, COLORS > 15 ? 15 : COLOR_WHITE);
        attron(COLOR_PAIR(1));
        int ok;
        // wypisujemy wiadomość od drugiego usera
        print_and_clear_screen_if_needed("\nReceived From User2: ");
        print_and_clear_screen_if_needed(shared_memory->buff2);
        // "zerujemy" bufor tego usera, żeby ponownie był pusty
        shared_memory->buff2[0] = '\0';
        memset(shared_memory->buff2, 0, 100);
        shared_memory->size2 = 0;
        attroff(COLOR_PAIR(1));
        attron(COLOR_PAIR(2));
        refresh();
        // ponawiamy label użytkownika
        print_and_clear_screen_if_needed("\nUser1: ");
        // gdyby user wcześniej zaczął coś pisać, ale
        // nie zdążył tego wysłać i jednocześnie inny user
        // przerwał ten proces, przywracamy na ekran tę wiadomość
        printw(local_buffer);
        refresh();
    }
}

int main() {

    // pobieramy identyfikator procesu
    // proszę zwrócić uwagę, że nie wywołano
    // fork; jednak ponieważ każdy proces ma pid
    // to możemy je odczytać
    int pid = getpid();

    // identyfikator bloku pamięci
    int shmid;

    // arbitralnie dobrany identyfikator bloku pamięci, wspólny
    // dla obydwu procesów
    int key = 123456;

    // tworzymy pamięć współdzieloną, do odczytu i zapisu
    shmid = shmget(key, sizeof(struct memory), IPC_CREAT | 0666);

    // dowiązujemy do wskaźnika shared_memory wcześniej zaalokowany blok pamięci
    shared_memory = (struct memory*)shmat(shmid, NULL, 0);

    // zapisujemy w strukturze identyfikator tego procesu
    // oraz status
    shared_memory->pid1 = pid;
    shared_memory->size = 0;

    // definiujemy handler dla sygnału SIGUSR1
    signal(SIGUSR1, handler);

    // tutaj tworzymy "okno" zarządzane przez bibliotekę ncurses
    WINDOW *w;
    char c;
    if (can_change_color() && COLORS >= 15) {
        init_color(BRIGHT_WHITE, 1000, 1000, 1000);
    }
```

```
    if (!can_change_color()) {
        printw("unfortunately, you can't change color!");
    }

    // inicjujemy ekran
    w = initscr();
    // wejście będzie pobierane w trybie nieblokującym
    nodelay(w, 1);
    // wyłączamy domyślne zachowanie terminala
    raw();
    noecho();
    wclrtoeol(w);
    // piszemy label usera
    // biblioteka ncurses ma funkcję printw od print window
    // z taką samą sygnaturą jak printf
    printw("%s", "User1: ");
    // alokujemy lokalny bufor, do którego będziemy pobierać wejście
    local_buffer = malloc(100 * sizeof(char));
    int size = 0;
    for (;;) {
        // pobieramy pojedynczy znak ze standardowego wejścia
        c = getch();
        // jeżeli wciśnięto esc albo znak q od quit kończymy działanie
        // programu
        if (c == 'q' || (int) c == 27) {
            shmdt((void*)shared_memory);
            // usuwamy blok pamięci współdzielonej
            shmctl(shmid, IPC_RMID, NULL);
            break; // wyjście z pętli nieskończonej, koniec wykonywania programu
        }
        // jeżeli odczytano w ogóle jakiś znak i nie jest to klawisz enter
        // dodajemy go do bufora
        if (c != ERR && (int) c != 10)
        {
            addch(c); // wypisujemy znak na ekran
            refresh();
            //shared_memory->buff1[shared_memory->size] = c;
            local_buffer[size] = c;
            size++;
            // shared_memory->size++;

        }

        // jeżeli wciśnięto enter (return)
        if ((int) c == 10) {
            local_buffer[size] = '\n';
            // wypisuejmu label usera
            memcpy(shared_memory->buff1, local_buffer,size);
            size = 0;
            memset(local_buffer, 0, 100);
            int ok = addstr("\nUser1: ");
            while (ok == ERR) {
                clear();
                ok = addstr("\nUser1: ");
            }
            refresh();
            // przesyłamy sygnał do drugiego procesu
            kill(shared_memory->pid2, SIGUSR2);
        }

    }
    // odświeżamy terminal
    refresh();
    // zwracamy sterowanie terminalem do stanu wyjściowego
    // czyli biblioteka ncurses przestaje kontrolować terminal
    endwin();
}
```

Plik chat_1_non_blocking.c

```
#include <ncurses.h> // biblioteka ncurses = new courses, da nam możliwość nieblokującego korzystania
// ze strumieni wejściowych
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>
#include <signal.h>
```

```c
#include <string.h>
#include <malloc.h>

#define BRIGHT_WHITE  15

char* local_buffer;

// struktura w której
// będziemy przechowywać
// 2 bufory, identyfikatory procesów i stan
// gotowości procesów
struct memory {
    char buff1[100];
    char buff2[100];
    int size, size2;
    int status, block, pid1, pid2;
};

// wskaźnik, w którym będziemy
// przechowywać strukturę
struct memory* shared_memory;

// pomocnicza funkcja, która wyczyści okno w razie
// gdy zabraknie w nim miejsca
void print_and_clear_screen_if_needed(char* s) {
    int ok = printw(s);
    if (ok == ERR) {
        clear();
        printw(s);
    }
}

void handler(int signum)
{
    // nadpisujemy handler dla sygnału
    // SIGUSR2, będzie to wiadomość
    // od pierwszego użytkownika czatu
    // wiadomość będzie wysyłał inny proces
    if (signum == SIGUSR2) {
        // wiadomość od drugiego użytkownika
        // chcemy napisać w innym kolorze
        start_color();
        init_pair(1, COLOR_BLUE, COLORS > 15 ? 15 : COLOR_WHITE);
        init_pair(2, COLOR_BLACK, COLORS > 15 ? 15 : COLOR_WHITE);
        attron(COLOR_PAIR(1));
        int ok;
        print_and_clear_screen_if_needed("\nReceived From User1: ");
        // wypisujemy wiadomość od drugiego usera
        printw(shared_memory->buff1);
        attroff(COLOR_PAIR(1));
        attron(COLOR_PAIR(2));
        // "zerujemy" bufor tego usera, żeby ponownie był pusty
        shared_memory->buff1[0] = '\0';
        memset(shared_memory->buff1, 0, 100);
        // ponawiamy label użytkownika
        print_and_clear_screen_if_needed("\nUser2: ");
        // gdyby user wcześniej zaczął coś pisać, ale
        // nie zdążył tego wysłać i jednocześnie inny user
        // przerwał ten proces, przywracamy na ekran tę wiadomość
        printw(local_buffer);
    //    shared_memory->block = 0;
    }
}

int main() {

    // pobieramy identyfikator procesu
    // proszę zwrócić uwagę, że nie wywołano
    // fork; jednak ponieważ każdy proces ma pid
    // to możemy je odczytać
    int pid = getpid();

    // identyfikator bloku pamięci
    int shmid;

    // arbitralnie dobrany identyfikator bloku pamięci, wspólny
    // dla obydwu procesów
    int key = 123456;
```

```
    // tworzymy pamięć współdzieloną, do odczytu i zapisu
    shmid = shmget(key, sizeof(struct memory), IPC_CREAT | 0666);

    // dowiązujemy do wskaźnika shared_memory wcześniej zaalokowany blok pamięci
    shared_memory = (struct memory*)shmat(shmid, NULL, 0);

    // zapisujemy w strukturze identyfikator tego procesu
    // oraz status
    shared_memory->pid2 = pid;
    // shared_memory->status = 0;
    shared_memory->size2 = 0;
    // shared_memory->block = 0;

    // definiujemy handler dla sygnału SIGUSR2
    signal(SIGUSR2, handler);

    // tutaj tworzymy "okno" zarządzane przez bibliotekę ncurses
    WINDOW *w;
    char c;

    // inicjujemy ekran
    w = initscr();
    // wejście będzie pobierane w trybie nieblokującym
    nodelay(w, 1);
    // wyłączamy domyślne zachowanie terminala
    raw();
    noecho();
    //timeout(3000);
    wclrtoeol(w);
    local_buffer = malloc(100 * sizeof(char));
    int size = 0;
    // piszemy label usera
    printw("%s", "User2: ");
    for (;;) {
        // pobieramy pojedynczy znak ze standardowego wejścia
        c = getch();
        // jeżeli wciśnięto esc albo znak q od quit kończymy działanie
        // programu
        if (c == 'q' || (int) c == 27) {
            shmdt((void*)shared_memory);
            // usuwamy blok pamięci współdzielonej
            // robi to tylko jeden proces
            shmctl(shmid, IPC_RMID, NULL);
            break; // wyjście z pętli nieskończonej, koniec wykonywania programu
        }
        // jeżeli znak nie jest znakiem sterującym
        // dodajemy go do bufora
        if (c != ERR && (int) c != 10)
        {
            addch(c); // wypisujemy znak na ekran
            local_buffer[size] = c;
            size++;
            //shared_memory->buff2[shared_memory->size2] = c;
            //shared_memory->size2++;
        }
        // jeżeli wciśnięto enter (return)
        if ((int) c == 10) {

            local_buffer[size] = '\n';
            memcpy(shared_memory->buff2, local_buffer,size);
            shared_memory->size2 = 0;
            size = 0;
            memset(local_buffer, 0, 100);
            // wypisuejmu label usera
            print_and_clear_screen_if_needed("\nUser2: ");
            // przesyłamy sygnał do drugiego procesu
            kill(shared_memory->pid1, SIGUSR1);
        }
        refresh();
    }
    // odświeżamy terminal
    refresh();
    // zwracamy sterowanie terminalem do stanu wyjściowego
    // czyli biblioteka ncurses przestaje kontrolować terminal
    endwin();
}
```

Plik chat_2_non_blocking.c

Tutaj w przeciwieństwie do poprzedniego przykładu, istnieje znacznie mniej założeń wstępnych, które muszą być spełnione, aby programy współdziałały ze sobą i były użyteczne. Przede wszystkim zastosowano bibliotekę ncurses (new curses), która jest rozwinięciem bardzo kiedyś popularnej biblioteki curses. Głównym przeznaczeniem tej biblioteki jest tworzenie bardziej zaawansowanych wierszowych interfejsów użytkownika. W poniższym przykładzie wykorzystamy ją z jednej strony do pokolorowania czcionki w terminalu (*tutaj uwaga techniczna: jeżeli osobiście korzystasz z innego motywu terminala, można zmienić zastosowane w przykładzie kolory*), jednak najważniejszą funkcjonalność, jaka zostanie dostarczona przez bibliotekę jest możliwość wykorzystania **nieblokującego wejścia w terminalu. Wejście blokujące to takie, które będzie oczekiwać (potencjalnie nieskończenie długo), na otrzymanie danych, zatrzymując dalsze wykonanie procesu. Wejście nieblokujące to takie, które jest pod tym względem bardziej elastyczne. Oznacza to, że dane mogą napłynąć (i jeżeli to się stanie zostaną zebrane), jednakże jeżeli nie napłyną w jakimś konkretnym momencie, to program będzie kontynuował swoje wykonanie.** Umożliwi nam to, że pomimo **wykonywania pętli niekończonej, która oczekuje na wejście wprowadzone z klawiatury, czynność ta może zostać przerwana przez obsługę zdarzenia,** polegającego na wypisaniu wiadomości od innego użytkownika. W przypadku biblioteki ncurses rozmiar terminala jest skończony i w przypadku gdy tego miejsca braknie, w przykładzie okno zostaje wyczyszczone; można uznać to rozwiązanie za nieidealne, ale odpowiednie dla przykładowego programu. Wykorzystanie biblioteki ncurses sprawiło, że musieliśmy samodzielnie zacząć zajmować się przetwarzaniem strumienia wejściowego. Można to podsumować w ten sposób, że chcąc mieć bardziej złożone, niestandardowe zachowanie terminala, adekwatne do naszego przypadku użycia, musieliśmy zastosować dedykowane, bardziej niskopoziomowe rozwiązanie. Dla ulepszenia działania programu, program nie zapisuje danych bezpośrednio do pamięci współdzielonej, ale do lokalnego bufora. **Technicznie można byłoby pracować bezpośrednio z pamięcią współdzieloną, jednakże zastosowane rozwiązanie sprawia, że ograniczamy pracę z tą pamięcią do minimum dla uzyskania celów wydajnościowych. Również ograniczanie korzystania z zasobu współdzielonego do minimum, ogranicza potencjalne błędy w synchronizacji działania tych programów, bądź wystąpienie tzw. race condition na tym zasobie.**

```
gcc -o chat_1_non_blocking chat_1_non_blocking.c -lncurses
```

Kompilacja programu wraz z zalinkowaniem biblioteki ncurses (biblioteka musi zostać najpierw zainstalowana jako pakiet Linuxa np. https://www.cyberciti.biz/faq/linux-install-ncurses-library-headers-on-debian-ubuntu-centos-fedora/).

- **Uruchom obydwa programy i wykonaj następujący scenariusz: zacznij pisać słowo „cześć” w jednym terminalu, ale nie naciśnij klawisza enter. Z drugiego terminala wpisz „witam” i wciśnij enter. Jakie jest zachowanie obydwu programów?**
- **Czym różni się to rozwiązanie, od tego z poprzedniego punktu? Które z nich Twoim zdaniem jest lepsze?**
- **Zauważ, że obecnie nie można wymazać danych za pomocą klawisza backspace. Dodaj taką funkcjonalność (za pomocą edycji danych w buforze i na ekranie).**
- **Zastanów się co mogłoby się stać, gdyby nie zastosowano jednak lokalnych buforów, tylko polegano jedynie na tych przechowywanych w pamięci współdzielonej.**

5. W jednym oknie terminala wpisz polecenia:

```
mkfifo /tmp/my_first_named_pipe
cat /tmp/my_first_named_pipe
```

W drugim oknie terminala wpisz polecenia:

```
Cat > /tmp/my_first_named_pipe
Hello world named pipe!
ctrl+c
```

Następnie przepisz poniższe programy:

```c
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
    int fd;
    char * myfifo = "/tmp/my_first_named_pipe";

    // utwórz named pipe z odpowiednimi flagami
    mkfifo(myfifo, 0666);

    fd = open(myfifo, O_WRONLY);
    char* buffer = "Hello world named pipe!";
    write(fd, buffer, sizeof("Hello world named pipe!"));
    close(fd);

    // usuwamy plik tymczasowy
    unlink(myfifo);

    return 0;
}
```

named_pipe_1.c

```c
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

#define MAX_BUF 1024

int main()
{
    int fd;
    char * myfifo = "/tmp/my_first_named_pipe";
    char buf[MAX_BUF];
    // czytamy z named pipe'a
    fd = open(myfifo, O_RDONLY);
    read(fd, buf, MAX_BUF);
    printf("Received: %s\n", buf);
    close(fd);

    return 0;
}
```

named_pipe_2.c

Realizują one dokładnie powyższe polecenia w terminalu, ale programowo. Warto zaznaczyć, że named pipe działa na zasadzie kolejki FIFO, stąd nazwa polecenia.

- **Czym named pipe różni się od poznanego dotychczas na zajęciach pipe’a (nazywanego dla odróżnienia unnamed)?**

6*. Przepisz przykład z punktu 4. w ten sposób, że zamiast z dwóch buforów niech korzysta z dwukierunkowego named pipe’a (najprościej uzyskać to za pomocą wykorzystania 2 niezależnych named pipeów). Zaleca się iteracyjne podejście, to jest najpierw napisanie programu blokującego, po czym jego ulepszenie do wersji nieblokującej.

7*. Zaimplementuj chat, który korzysta z message queve, gdzie komunikat pierwszego klienta do drugiego ma typ 1, a komunikat drugiego klienta do pierwszego ma typ 2. Dla usprawnienia komunikacji wykorzystaj sygnały oraz nieblokujące odczytywanie komunikatów.

*** - zadanie dla osób chętnych**